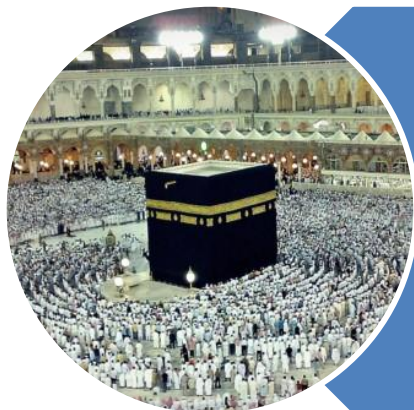


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

# ARRAYS

## Introduction

**“No Bonus” marks for this course anymore**

C

P

C

S

2

0

4

3

## Motivation

- You want to store 5 numbers in a program
  - No problem. You define five int variables:  
`int num1, num2, num3, num4, num5;`
  - Easy enough, right?
  - But what if you want to store 1000 numbers?
    - Are you really going to make 1000 separate variables?  
`int num1, num2, ..., num998, num999, num1000;`
    - That would be CRAZY!
- So what is the solution?
  - A data structure! Specifically, an array!
    - An array is one of the most common data structures.

C

P

C

S

2

0

4

4

## Basic Concepts

- Array name (data)
- Index/subscript (0...9)
- The slots are numbered sequentially starting at zero (Java, C++)
- If there are N slots in an array, the index will be 0 through N-1
- Array length =  $N = 10$
- Array size =  $N \times \text{Size of an element} = 40$

|   | data |
|---|------|
| 0 | 23   |
| 1 | 38   |
| 2 | 14   |
| 3 | -3   |
| 4 | 0    |
| 5 | 14   |
| 6 | 9    |
| 7 | 103  |
| 8 | 0    |
| 9 | -56  |

C

P

C

S

2

0

4

5

## Using Arrays

- Array\_name[index]
- For example, in Java
  - System.out.println(data[4]);
    - will display 0
  - data[3] = 99;
    - Will replace -3 with 99

| data |     |
|------|-----|
| 0    | 23  |
| 1    | 38  |
| 2    | 14  |
| 3    | -3  |
| 4    | 0   |
| 5    | 14  |
| 6    | 9   |
| 7    | 103 |
| 8    | 0   |
| 9    | -56 |

| data |     |
|------|-----|
| 0    | 23  |
| 1    | 38  |
| 2    | 14  |
| 3    | 99  |
| 4    | 0   |
| 5    | 14  |
| 6    | 9   |
| 7    | 103 |
| 8    | 0   |
| 9    | -56 |

## Using Arrays

- `data[ -1 ]`
  - ❖ What will be the output of?
    - ❖ `data[5] + 10`
    - ❖ `data[3] = data[3] + 10`
  - illegal
- `data[ 10 ]`
  - illegal ( $10 > \text{upper bound}$ )
- `data[ 1.5 ]`
  - illegal
- `data[ 0 ]`
  - OK
- `data[ 9 ]`
  - OK

| data |     |
|------|-----|
| 0    | 23  |
| 1    | 38  |
| 2    | 14  |
| 3    | -3  |
| 4    | 0   |
| 5    | 14  |
| 6    | 9   |
| 7    | 103 |
| 8    | 0   |
| 9    | -56 |

C

P

C

S

2

0

4

7

## Array Dimensionality

- One dimensional (just a linear list)

|   |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|
| 5 | 10 | 18 | 30 | 45 | 50 | 60 | 65 | 70 | 80 |
|---|----|----|----|----|----|----|----|----|----|

- Only one subscript is required to access an individual element

- Two dimensional (matrix or table)

|       | Col 0 | Col 1 | Col 2 | Col 3 |
|-------|-------|-------|-------|-------|
| Row 0 | 20    | 25    | 60    | 40    |
| Row 1 | 30    | 15    | 70    | 90    |

## 2D Arrays

- Given the following array (whose name is 'M')

|    |    |    |    |
|----|----|----|----|
| 20 | 25 | 60 | 40 |
| 30 | 15 | 70 | 90 |

- Two indices/subscripts are now required to reference a given cell
- M[0][0] ?
- M[1][2] ?
- M[3][4] ?



C

P

C

S

2

0

4

## Array Declaration

- You declare an array as follows:

```
int[] grades;
```

- This simply makes a an array variable (grades)
  - But it does NOT specify where that variable refers to
- We can also declare the following:

```
int[] grades = new int[10];
```

  - Now the array variable grades refers to an array of ten integers
    - By default, numeric elements are initialized to zero

C

P

C

S

2

0

4

## Memory and Initialization

- When the array is created, memory is reserved for its contents
- You can also specify the values for the array instead of using the new operator
- Example:

```
int[] grade = {95, 93, 88}; //array of  
3 ints
```

- To find the length of an array, use the length method

```
int numGrades = grade.length();
```

C

P

C

S

2

0

4

## Arrays' Properties in Java

- Arrays are objects
- Arrays are created dynamically, at run time
- An array type variable holds a reference to the array object
- An array's length is set when the array is created, and it cannot be changed.
- Arrays can be duplicated with the **Object.clone()** method
- An **ArrayIndexOutOfBoundsException** is thrown if index exceeds array length

C

P

C

S

2

0

4

## Array Processing in JAVA

- **java.util.Arrays** class provides many built-in functions for sorting and searching
  - The method **Arrays.sort(array\_name)** is used to sort an array
  - The method **Arrays.binarySearch(array\_name, target)** is used to search through a sorted array
  - The method **Arrays.equals(array1, array2)** is used to check if two arrays are equal
    - Meaning, if the same values are at each index position

**"No Bonus" marks for this course anymore**

## Array Processing in Java

```

1  import java.util.Arrays;
2
3  public class TestArrays {
4      public static void main(String[] args) {
5          int[] a = {44, 77, 55, 22, 99, 88, 33, 66};
6          print(a); 44 77 55 22 99 88 33 66
7          Arrays.sort(a);
8          print(a); 22 33 44 55 66 77 88 99
9          int k = Arrays.binarySearch(a, 44);
10         System.out.printf("Arrays.binarySearch(a, 44): %d\n", k);
11         System.out.printf("a[%d]: %d\n", k, a[k]);
12         k = Arrays.binarySearch(a, 45);
13         System.out.printf("Arrays.binarySearch(a, 45): %d\n", k);
14         int[] b = new int[8];
15         print(b);
16         Arrays.fill(b, 55);
17         print(b);
18         System.out.println("Arrays.equals(a,b): " + Arrays.equals(a,b));
19     }
20 }

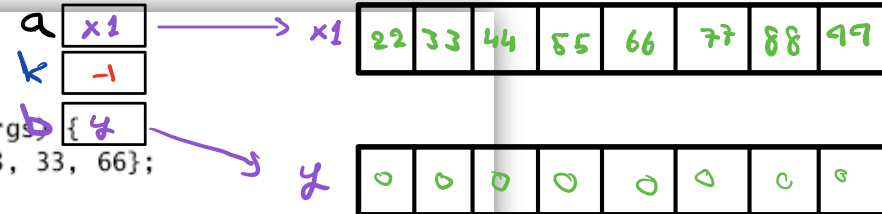
```

The output is

```

44 77 55 22 99 88 33 66
22 33 44 55 66 77 88 99
Arrays.binarySearch(a, 44): 2
a[2]: 44
Arrays.binarySearch(a, 45): -1
0 0 0 0 0 0 0 0
55 55 55 55 55 55 55 55
Arrays.equals(a,b): false

```



# ARRAY

## As a Data Structure

C

P

C

S

2

0

4

## What is an Array?

- An array is a data structure
  - It is a collection of multiple values of same type
  - Examples:
    - An array of student grades
    - An array of student names
    - An array of objects (OOP perspective!)
- **Assumption:** Data in array is always sorted
- Array elements are initialized with “ $+\infty$ ”

C

P

C

S

2

0

4

## Array Characteristics

- **Homogeneity**

- All elements in an array must have the same data type

- **Contiguous Memory**

- Array elements (or their references) are stored in contiguous/consecutive memory locations

- **Direct access to an element**

- Index reference is used to access it directly

- **Static data structure**

- An array cannot grow or shrink during program execution...the size is fixed



# ARRAY

## Operations

## Array Operations

- **Accessing/indexing** an element using its index
  - Performance is very fast
  - We can access an index immediately without searching
  - `myArray [1250] = 55;`
    - we immediately access array spot 1250 of myArray

```
public static void access(double[] a, int index) {  
  
    a[index] = a[index] + a[index] * 0.20;  
  
    System.out.println ("Updated value "+ a[index]);  
}
```

C

P

C

S

2

0

4

## Array Operations

- **Traversing an array**
  - Display all contents of an array
    - All elements will be displayed
    - Every elements will be displayed exactly once

```
public static void display(int[] a) {  
    for (int i=0; i<a.length; i++)  
        System.out.println (a[i]);  
}
```

C

P

C

S

2

0

4

## Array Operations

- **Insertion:** add an element at a certain index
  - What if we want to add an element at the beginning?
    - This would be a very slow operation! Why?
      - Because we would have to shift ALL other elements over one position
    - What if we add an element at the end?
      - It would be FAST. Why? No need to shift.

```
public static void insert(int[] a, int value) {
    if (a . length == 0 || value < a[0]){
        for (int i = a.length-1; i > 0; i--)
            a[i] = a[i-1];
        a[i] = value;
    }
    else{
        int j = 0;
        while ( j < a . length -1){
            if (a[j+1] > value)
                break;
            j++;
        }
        for (int k = a.length-1; k > j+1; k--)
            a[k] = a[k-1];
        if ( j == a.length-1)
            System. out. println("Array Full");
        else
            a[k] = value;
    }
}
```

C

P

C

S

2

0

4

## Array Operations

- **Deletion:** remove an element at a certain index
  - Remove an element at the beginning of the array
    - Performance is again very slow.
      - Because ALL elements need to shift one position backwards
  - Remove an element at the end of an array
    - Very fast because of no shifting needed

```
public static void delete(int[] a, int value) {
    if (value == a[0]){
        for (int i = 1; i < a . length 0; i++)
            a[i-1] = a[i];

        a[i-1] = + ∞; // maximum value of domain range
    }
    else{
        int j = 0;
        while ( j < a . Length-1){
            if (a[j+1] == value){
                for (int k = j+1; k < a . Length-1; k++)
                    a[k] = a[k+1];

                a[k] = + ∞;
                break;
            }
            j++;
        }
        if ( j == a . Length - 1)
            System.out.println ("Value is not found");
    }
}
```





C

P

C

S

2

0

4

## Array Operations

- **Searching** through the array:
  - Depends on the algorithm
  - Some algorithms are faster than others
    - More detail coming soon!
  - Linear Search
  - Binary Search

## Linear Search

C

P

C

S

2

0

4

## Linear Search

```
public static boolean search(int[] a, int value) {  
    for (int i=0; i<a.length; i++) {  
        if (a[i] == value) {  
            return true;  
        }  
    }  
    return false;  
}
```

- The code returns a Boolean value
  - If found it will return **TRUE**
  - Otherwise **FALSE**

## Linear Search

```
public static int search(int[] a, int value) {  
    for (int i=0; i<a.length; i++) {  
        if (a[i] == value) {  
            return i;  
        }  
    }  
    return -1;  
}
```

- The code returns location of the values
  - If found it will return **Index** of the value
  - Otherwise it will return **-1**

# Linear Search

## Analysis

C

P

C

S

2

0

4

## Analysis of Linear Search

- Applicable for all data i.e. Sorted or Unsorted
  - Basic operation is **“comparison”**
  - They ONLY way to be sure that a value isn't in the array is to **look at every single spot** of the array
  - If we have **100 elements** then we have to make **100 comparisons** to be sure about the value
  - Therefore for **“n”** elements, the number of comparisons will be “n” i.e.  **$O(n)$**

# Binary Search

C

P

C

S

2

0

4

## Binary Search (Introduction)

- Number Guessing Game from childhood
  - Remember the game you most likely played as a child
    - I have a secret number between 1 and 100.
    - Make a guess and I'll tell you whether your guess is too high or too low.
    - Then you guess again. The process continues until you guess the correct number.
    - Your job is to MINIMIZE the number of guesses you make.



**“No Bonus” marks for this course anymore**

## Binary Search (Introduction)

- Number Guessing Game from childhood
  - What is the first guess of most people?
    - 50.
  - Why?
    - No matter the response (too high or too low), the most number of possible values for your remaining search is 50 (either from 1-49 or 51-100)
    - Any other first guess results in the risk that the possible remaining values is greater than 50.
      - Example: you guess 75
      - I respond: too high
      - So now you have to guess between 1 and 74
        - » 74 values to guess from instead of 50

## Binary Search (Introduction)

- Applicable only on **Sorted array**

| index | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8   |
|-------|---|---|----|----|----|----|----|----|-----|
| value | 2 | 6 | 19 | 27 | 33 | 37 | 38 | 41 | 118 |

- We are searching for the value, 19
- So where is halfway between?
  - One guess would be to look at 2 and 118 and take their average (60).
  - But 60 isn't even in the list
  - And if we look at the number closest to 60
    - It is almost at the end of the array

C

P

C

S

2

0

4

## Binary Search (Introduction)

- We quickly realize that if we want to adapt the number guessing game strategy to searching an array, we **MUST** search in the middle INDEX of the array.

| index | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8   |
|-------|---|---|----|----|----|----|----|----|-----|
| value | 2 | 6 | 19 | 27 | 33 | 37 | 38 | 41 | 118 |

- In this case:
  - The lowest index is 0
  - The highest index is 8
  - So the middle index is 4

## Binary Search (Introduction)

| index | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8   |
|-------|---|---|----|----|----|----|----|----|-----|
| value | 2 | 6 | 19 | 27 | 33 | 37 | 38 | 41 | 118 |

- We would ask, “is the number I am searching for, 19, greater or less than the number stored in index 4?”
  - Index 4 stores 33
- The answer would be “less than”
- So we would modify our search range to in between index 0 and index 3
  - Note that index 4 is no longer in the search space
- We then continue this process
  - The second index we’d look at is index 1, since  $(0+3)/2=1$
  - Then we’d finally get to index 2, since  $(2+3)/2 = 2$
  - And at index 2, we would find the value, 19, in the array

## Binary Search (Code return boolean)

```
public static boolean binSearch(int[] a, int value) {  
    int low = 0;  
    int high = a.length-1;  
    while (low <= high) {  
        int mid = (low + high)/2;  
        if (value == a[mid])  
            return true; // meaning, "found"  
        else if (value < a[mid])  
            high = mid - 1;  
        else // if (value > a[mid])  
            low = mid + 1;  
    }  
    return false; // this only happens if not found  
}
```

C

P

C

S

2

0

4

## Binary Search (Code return index)

```
public static int binSearch(int[] a, int value) {  
    int low = 0;  
    int high = a.length-1;  
    while (low <= high) {  
        int mid = (low + high)/2;  
        if (value == a[mid])  
            return mid; // the index of value  
        else if (value < a[mid])  
            high = mid - 1;  
        else // if (value > a[mid])  
            low = mid + 1;  
    }  
    return -1; // this only happens if not found  
}
```

# Binary Search

## Analysis

C

P

C

S

2

0

4

## Analysis of Binary Search

- Let's analyze how many comparisons (guesses) are necessary when running this algorithm on an array of  $n$  items

First, let's try  $n = 128$

- After 1 guess, we have 64 items left,
- After 2 guesses, we have 32 items left,
- After 3 guesses, we have 16 items left,
- After 4 guesses, we have 8 items left,
- After 5 guesses, we have 4 items left,
- After 6 guesses, we have 2 item left
- After 7 guesses, we have 1 items left.
- After 8 guesses, we have 0 items left.



C

P

C

S

2

0

4

## Analysis of Binary Search

- General case for  $n$  items
  - After 1 guesses, we have  $n/2$  items left,
  - After 2 guesses, we have  $n/4$  items left,
  - After 3 guesses, we have  $n/8$  items left,
  - After 4 guesses, we have  $n/16$  items left,
  - .....
  - .....
  - .....
  - So on until we have **1** item left

## Analysis of Binary Search

- General case for  $n$  items
  - After 1 guesses, we have  $n/2$  items left,  $n/2^1$
  - After 2 guesses, we have  $n/4$  items left,  $n/2^2$
  - After 3 guesses, we have  $n/8$  items left,  $n/2^3$
  - After 4 guesses, we have  $n/16$  items left,  $n/2^4$
  - .....
  - After 10 guesses, we have  $n/2^{10}$
  - After  $k$  guesses, we have  $n/2^k$
  - We will stop when we left with **1** item

C

P

C

S

2

0

4

## Analysis of Binary Search

- So we will stop once

$$\frac{n}{2^k} = 1 \quad \longrightarrow \quad n = 2^k \quad \longrightarrow \quad k = \log_2 n$$

- This means that a binary search roughly takes  **$\log_2 n$**  comparisons when searching in a sorted array of  **$n$**  items
- Efficiency of Binary Search is  **$O(\log_2 n)$**

C

P

C

S

2

0

4

## Linear Search **vs** Binary Search

- Linear search  **$O(n)$**
- Binary Search  **$O(\log_2 n)$**
- **Binary Search** is more efficient

| <u><b>n</b></u> | <u><b>log n</b></u> |
|-----------------|---------------------|
| 8               | 3                   |
| 1024            | 10                  |
| 65536           | 16                  |
| 1048576         | 20                  |
| 33554432        | 25                  |
| 1073741824      | 30                  |

# List Matching Problem

C

P

C

S

2

0

4

## List Matching Problem

- You are given two lists of Last Names
  - Within each list, all names are distinct
  - Also, each list is unsorted / sorted
- Problem:
  - Output the names common to both lists

C

P

C

S

2

0

4

## Sorted List Matching Problem

- **Solution 01 (Brute Force – Unsorted lists)**
  - For each name on list #1, do the following:
    - a) Search for the current name in list #2
    - b) If the name is found, output it.
  - Steps a and b are run for each of the **n** names in List #1, resulting in an  **$O(n^2)$**  running time.

C

P

C

S

2

0

4

## Sorted List Matching Problem

- **Solution 01 (Brute Force - Unsorted lists)**

```
public static void printMatches(String[] lst1, String[] lst2){  
    int i, j;  
  
    for (i=0; i < lst1.length; i++) {  
  
        for (j=0; j < lst2.length; j++) {  
  
            if (lst1[i].compareTo(lst2[j])==0) {  
  
                System.out.println(lst1[i]);  
                break;  
            }  
        }  
    }  
}
```



C

P

C

S

2

0

4

## Sorted List Matching Problem

- **Solution 02 (One Sorted List)**

- Compare the target to the middle item in the list.
- If the target is the same as the middle item
  - you've found the target.
- If it's before the middle item
  - repeat this procedure on the items before the middle.
- If it's after the middle item
  - repeat on the items after the middle.
- The method halves the number of items to check each time. So It runs in logarithmic time:  **$O(\log n)$**
- Matching  **$n$**  items of list 1 will take  **$O(n \log n)$**

C

P

C

S

2

0

4

## Sorted List Matching Problem

- **Solution 02 (One Sorted List)**

```
public static void printMatches(String[] lst1, String[] lst2){  
  
    int i;  
  
    for (i=0; i < lst1.length; i++) {  
  
        if (binSearch(lst2, lst1[i]))  
  
            System.out.println(lst1[i]);  
    }  
}
```

C

P

C

S

2

0

4

## Sorted List Matching Problem

- **Solution 03 (Both Sorted Lists)**

- 1) Start two “markers”

- One for each list, at the beginning of both lists

- 2) Repeat the following steps until one marker has reached the end of the list

- a) Compare the two names that the markers are pointing at

- b) If they are equal,

- Output the name and advance BOTH makers one spot

- c) If they are NOT equal,

- Simply advance the marker pointing to the name that comes earlier, alphabetically, one spot

- Try coding this up on your own

C

P

C

S

2

0

4

## Sorted List Matching Problem

- Solution 03 (Both Sorted Lists)**

|   | <u>List #1</u> | <u>List #2</u> |   |
|---|----------------|----------------|---|
| → | Adams          | Boston         | ← |
| → | Bell           | Davis          | ← |
| → | Davis          | Duncan         | ← |
| → | Harding        | Francis        | ← |
| → | Jenkins        | Gamble         | ← |
| → | Lincoln        | Harding        | ← |
| → | Simpson        | Mason          | ← |
| → | Zoeller        | Simpson        | ← |
|   |                |                | ← |

C

P

C

S

2

0

4

## Sorted List Matching Problem

- **Solution 03 (Both Sorted Lists)**

- For each loop iteration, we advance at least one marker
- As such, the maximum number of iterations would be the total number of names on both lists, which is  $n$ , the length of both lists
  - For each iteration, we are doing a constant amount of work
  - Essentially a comparison and/or outputting a name
- Thus, this algorithm runs in about  $2n$  steps i.e.  **$O(n)$**

C

P

C

S

2

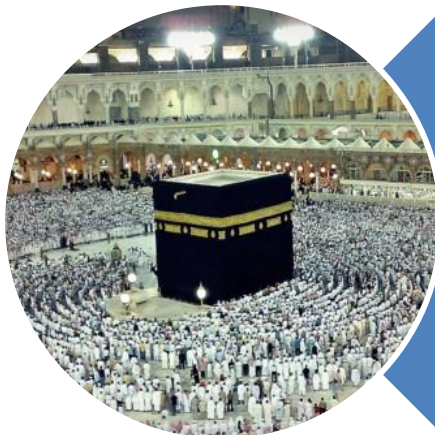
0

4

## Array Operations

- **Sorting:** Arranging the values in ascending or descending order
  - Insertion Sort
  - Selection Sort
  - Quick Sort etc.
- *This will be covered later on*





Allah (Alone) is Sufficient for us,  
and He is the Best Disposer of  
affairs (for us). **(Quran 3 : 173)**